

Bilgi Geometrisi ve Kodun Topolojisi: Yazılım Sistemlerini Yapısal Biçim Üzerinden Anlamak

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Modern dünyada karmaşıklık yalnızca fiziksel sistemlerde değil, bilgi sistemlerinde de ortaya çıkmaktadır. Yazılım mimarileri, organizasyon yapıları, karar mekanizmaları ve dijital platformlar giderek daha büyük ve birbirine bağlı yapılar haline gelmiştir. Bu dönüşümle birlikte kritik soru artık yalnızca “bir sistem çalışıyor mu” sorusu değildir; asıl mesele, “bir sistem anlaşılabilir, sürdürülebilir ve yönetilebilir mi” sorusudur. Çünkü modern sistemlerde başarısızlık çoğu zaman tek bir algoritmanın yetersizliğinden değil; bilginin dağınıklaşmasından, akışların belirsizleşmesinden ve yapının kontrol edilemez hale gelmesinden kaynaklanır.

Bu bağlamda “bilgi geometrisi” kavramı, bilginin sistem içindeki organizasyon biçimini anlamak için güçlü bir düşünsel çerçeve sunmaktadır. Bilginin nasıl aktığı, nasıl katmanlandığı, nasıl tekrar ettiği ve hangi yapılar etrafında organize olduğu; sistemin anlaşılabilirliğini ve ölçeklenebilirliğini doğrudan etkiler.

Bu ihtiyaç bir benzetmeyle açıklanabilir: herkes amatör düzeyde takım oyunu oynayabilir; ancak en üst düzey rekabette oynamak yalnızca bireysel yetenek değil, yüksek koordinasyon, strateji, disiplin ve sistem organizasyonu gerektirir. Yazılım sistemleri de benzer biçimde çalışır: küçük ölçekte ayakta kalan yapılar, karmaşıklık ve ölçek arttığında çoğu zaman kırılganlaşır. Teknik borç, belirsiz bilgi akışı ve bozuk sistem geometrisi; düşük yoğunlukta tolere edilebilirken, büyük sistemlerde ciddi performans ve sürdürülebilirlik problemlerine dönüşür.

Belirtmek gerekir ki “bilgi geometrisi” ifadesi burada, resmi ve yerleşik bir bilgisayar bilimi terimi olarak değil, bir sistemde bilginin nasıl organize olduğunu, nasıl aktığını ve farklı parçalar arasında nasıl bağlandığını anlatan güçlü bir metafor olarak kullanılmaktadır. Kavram temelde şu soruya yanıt arar:

Bir sistemde bilgi hangi biçimde düzenleniyor, hangi yollardan akıyor ve hangi ilişkiler aracılığıyla anlam kazanıyor?

Bir yazılım sistemi yalnızca satırlardan oluşan bir metin değildir; aynı zamanda veri akışları, bağımlılıklar, modüller, durum kaynakları, yan etkiler, testler, sözleşmeler ve alan kavramlarından oluşan ilişkisel bir yapıdır. Bu yapı, sistemin bilgi geometrisini oluşturur.

Bilginin Sistem İçindeki Organizasyonu

Bir sistemde bilginin nasıl konumlandığı, sistemin sağlıklı olup olmadığını büyük ölçüde belirler. Bilgi şu biçimlerde organize olabilir:

- dağınık,
- merkezi,
- tekrar eden,
- katmanlı,
- döngüsel,
- lokal,
- her yere sızan,
- kontrollü sınırlar içinde akan.

Bu özelliklerin her biri sistemin geometrisini etkiler. Örneğin aynı bilginin birden fazla yerde tekrar etmesi, sistemde fazladan bağlantılar ve belirsizlikler üretir. Bir durumun tek bir kaynağı varsa sistem daha kolay anlaşılır; aynı durum farklı yerlerde farklı biçimlerde tutuluyorsa bilgi geometrisi bulanıklaşır. Bu nedenle bilgi geometrisi, sistemin yalnızca teknik yapısını değil, aynı zamanda anlaşılabilirliğini de ifade eder.

İyi ve Kötü Bilgi Geometrisi

İyi Bilgi Geometrisi

İyi bir bilgi geometrisinde akış yönleri nettir, sorumluluklar ayrılmıştır ve sistemin hangi parçasının hangi işi yaptığı kolayca anlaşılır. Basit bir katmanlı arayüz mimarisi şu şekilde gösterilebilir:

```
UI
|
v
Facade
|
v
Service
|
v
API
```

Bu yapıda bilgi yönlü biçimde akar: arayüz doğrudan dış sistem detaylarını bilmez, facade katmanı arayüz ile alt katmanlar arasında sade bir yüzey sağlar, servis dış sistemlerle iletişim kurar. Bu mimarinin kazanımları şunlardır:

- arayüz yalnızca gösterim ve kullanıcı etkileşiminden sorumludur,
- facade kullanım niyetini sadeleştirir,
- servis dış iletişim detaylarını taşır,
- dış sistem sınırı nettir,
- veri akışı izlenebilirdir,
- değişikliklerin etki alanı lokal kalır.

Böyle bir yapıya hem insan geliştirici hem de yapay zekâ daha kolay uyum sağlar; çünkü sistemin anlam haritası açıktır.

Kötü Bilgi Geometrisi

Kötü bir bilgi geometrisinde sorumluluklar karışır, veri farklı yönlerde dağılır, bağımlılıklar görünmezleşir ve yan etkiler kontrolsüz biçimde yayılır. Aşağıdaki yapı problemlili bir geometriyi temsil eder:

```

UI <-> Service
|      ^
v      \
Store <-> Helper
^        |
\        v
Random util

```

Bu yapıda bilgi akışı net değildir: arayüz servis ile doğrudan konuşur, durum deposu birden fazla noktadan etkilenir, yardımcı yapılar sistem davranışının parçası haline gelir ve belirsiz araçlar sistemin farklı noktalarına kontrolsüz bağlantılar kurar. Ortaya çıkan problemler şunlardır:

- durum her yere yayılır,
- yan etkiler gizlenir,
- verinin kaynağı belirsizleşir,
- değişikliklerin etki alanı öngörülemez hale gelir,
- küçük bir değişiklik dalga etkisiyle başka bölgeleri bozar,
- test yazmak zorlaşır,
- yapay zekâ yanlış bağımlılık ilişkileri kurabilir.

Bu tür sistemler yalnızca karmaşık değildir; daha doğru bir ifadeyle, topolojik olarak çamurlaşmıştır:

Kötü mimaride problem yalnızca kodun uzun olması değildir. Problem, sistemin şeklinin okunamaz hale gelmesidir.

Topolojik Çamur

“Topolojik çamur” ifadesi, sistemdeki bağımlılıkların, veri akışlarının ve sorumlulukların artık sağlıklı biçimde ayrıştırılamadığı durumu anlatmak için önerilebilir. Bu durumda sistemde her şey her şeyle bağlantılı hale gelir: bir fonksiyonun yalnızca kendi işini yapıp yapmadığı belli değildir; bir bileşenin yalnızca arayüz mü çizdiği, yoksa iş mantığı mı çalıştırdığı, durumu mu değiştirdiği, önbelleği mi temizlediği veya başka bir olayı mı tetiklediği izlenemez.

Topolojik çamurun temel problemi şudur:

Sistemin herhangi bir noktasına dokunulduğunda, hangi noktaların etkileneceği önceden tahmin edilemez.

Bu, hem insan geliştirici hem de yapay zekâ için ciddi bir engeldir; çünkü doğru değişiklik yapabilmek için sistemin şeklinin okunabilir olması gerekir.

Karmaşıklığın Şekli

Karmaşıklık yalnızca miktar meselesi değildir; karmaşıklığın bir şekli vardır. Bir sistem büyük fakat düzenli, başka bir sistem daha küçük fakat çok daha karmaşık olabilir. Bu farkı belirleyen, sistemdeki parçaların nasıl bağlandığıdır. Bazı yapılar doğaları gereği daha yönetilebilirdir:

- ağaç yapıları yönetilebilirdir,
- modüler yapılar ölçeklenebilirdir,
- ağ yapıları güçlü fakat karmaşıktır,
- düğümlenmiş (spaghetti) yapılar kırılgandır.

Bu nedenle yazılımda yalnızca kod satırı sayısına bakmak yeterli değildir; asıl önemli olan, bu satırların nasıl bir ilişki yapısı oluşturduğudur.

Yazılım Dışındaki Bilgi Geometrileri

Bilgi geometrisi yalnızca yazılıma özgü bir düşünce değildir. Birçok doğal, sosyal ve kültürel sistem de benzer yapısal organizasyonlara sahiptir.

İnsan Vücudu

İnsan vücudu farklı geometrik ve ilişkisel organizasyonlardan oluşur: damar sistemi büyük ölçüde ağaç yapısına, sinir sistemi ağ yapısına benzer; hücreler modüler yapılardır; hormon sistemi ise akış dinamikleriyle çalışır. Damar sistemi dallanarak yayılır, sinir sistemi yoğun bağlantılar kurar, hücreler belirli sınırları olan modüller gibi işler ve hormon sistemi kimyasal sinyallerin akışıyla davranış üretir.

Şehirler

Şehirler de bilgi ve hareket geometrisine sahiptir: ızgara plan düzenli ve öngörülebilir bir yapı sunar; organik ortaçağ dokusu tarihsel olarak büyümüş, daha düzensiz bir yapıdır; merkez–çevre ulaşım modeli ise merkezi düğümler üzerinden dağılan bir düzeni temsil eder. Bir şehrin sokak yapısı, insanların nasıl hareket ettiğini, trafiğin nerede sıkıştığını ve şehrin nasıl deneyimlendiğini etkiler. Yazılım mimarisinde bağımlılık grafiği benzer bir rol üstlenir.

Müzik

Müzikte de geometrik ve ilişkisel organizasyon bulunur: tema bir merkez oluşturur, varyasyonlar bu merkezin etrafında gelişir, dönüşler yapısal bütünlük sağlar, gerilim ve çözülme müzikal hareketin yönünü belirler. Bir müzik parçası yalnızca notaların sıralanması değildir; gerilim, çözülme, tekrar, varyasyon ve dönüş ilişkileriyle anlam kazanır.

Anlatı

Hikâye anlatımı da belirli bir yapısal geometriye sahiptir: anlatı yayı, dallanan yollar, duygusal tempo, çatışma ve çözüm, karakter dönüşümü. İyi bir hikâyede olaylar rastgele dizilmez; duygusal akış, karakter gelişimi ve olay örgüsü belirli bir yapı içinde ilerler.

Bilim

Bilimsel bilgi de geometrik olarak düşünülebilir: bilgi grafikleri, anlamsal ağlar, kategori hiyerarşileri, nedensel modeller ve kuram yapıları. Bilim yalnızca veri biriktirme faaliyeti değildir; veriler arasındaki ilişkileri kurma, kavramları sınıflandırma ve açıklama modelleri oluşturma sürecidir.

Geometri Kavramının Buradaki Anlamı

Bu bağlamda geometri, dar anlamda yalnızca çizgi, açı, yüzey veya hacim anlamına gelmez. Daha geniş anlamda, bir yapının uzaysal ve ilişkisel organizasyonunu ifade eder:

Bir sistemin parçalarının birbirine göre nasıl konumlandığı, nasıl bağlandığı ve hangi yönlerde etkileştiği.

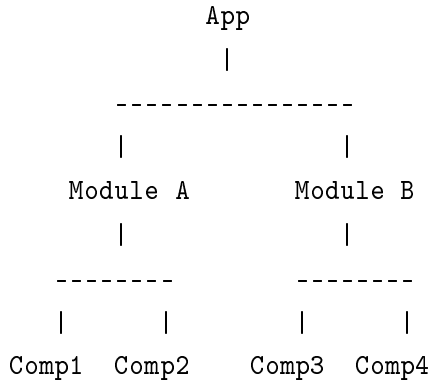
Bu açıdan bakıldığında yazılım sistemi de bir geometriye sahiptir: dosyalar, modüller, fonksiyonlar, bileşenler, servisler, durum kaynakları, testler ve dış sistem sınırları birlikte bir yapı oluşturur. Bu yapının biçimi sağlıklıysa sistem anlaşılır; biçimi bozursa sistem kırılganlaşır.

Yapısal Formların Sınıflandırılması

Kod tabanlarının aldığı tipik yapısal formlar, birer ideal tip olarak sınıflandırılabilir.

Ağaç (Tree) Geometrisi

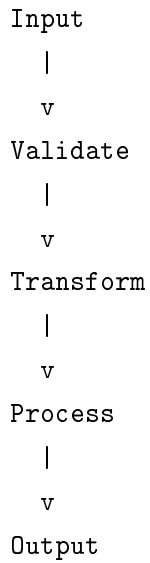
Ağaç geometrisi, hiyerarşik yapılarda görülür; üst seviyeden alt seviyeye doğru dallanma vardır:



Bu yapı, sorumlulukların üstten alta dağıldığı sistemlerde anlaşılır bir model sağlar.

Akış (Flow) Geometrisi

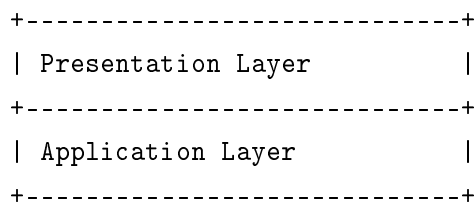
Akış geometrisi, verinin veya işlemin belirli bir sırayla ilerlediği sistemleri temsil eder:

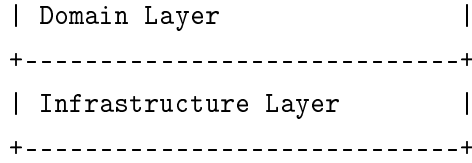


Bu form özellikle işlem hatları, form işleme, veri işleme ve sunucu tarafı iş akışı tasarımlarında kullanışlıdır.

Katmanlı (Layered) Geometri

Katmanlı geometri, yazılım mimarisinde en sağlıklı ve anlaşılır formlardan biridir:

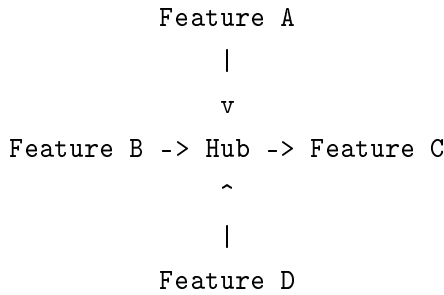




Her katman farklı bir sorumluluk taşır; üst katmanlar alt katmanları kullanır, fakat alt katmanlar üst katmanlara bağımlı olmamalıdır.

Merkez (Hub) Geometrisi

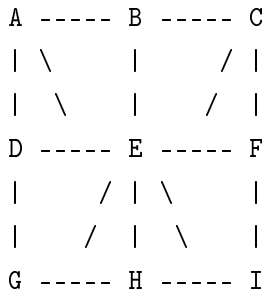
Merkez geometrisinde birçok parça tek bir düğüm üzerinden haberleşir:



Kontrollü kullanıldığında bu yapı koordinasyon sağlar; ancak merkez aşırı sorumluluk üstlenirse aşağıda incelenen merkezi şişme problemine dönüşür.

Ağ (Network) Geometrisi

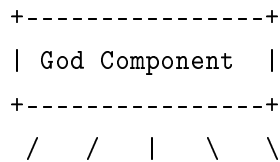
Ağ geometrisi, çok sayıda parçanın birbirine bağlı olduğu yapılarda görülür:

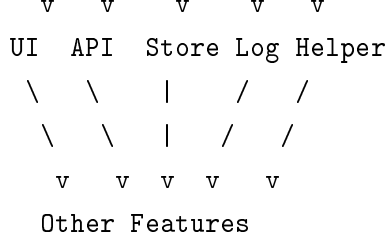


Bu form bazı alanlarda doğaldır: sosyal ağlar, bilgi grafikleri ve nöral bağlantılar ağ geometrisine sahiptir. Ancak yazılım mimarisinde kontrolsüz ağ yapısı karmaşıklık üretir.

Merkezi Şişme: God Component Geometrisi

Bazı sistemlerde tek bir bileşen veya servis aşırı büyür ve sistemin merkezi haline gelir:

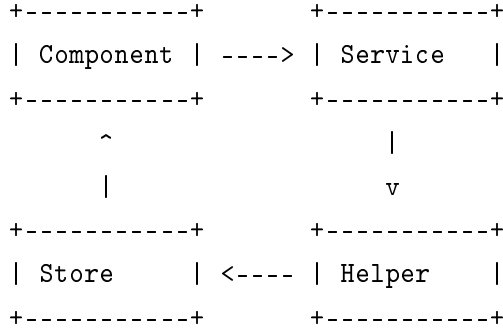




Bu geometri tehlikelidir; çünkü sistem tek bir merkezde aşırı yoğunlaşır ve bu merkez bozulduğunda veya değiştiğinde çok geniş bir alan etkilenir.

Döngüsel Bağımlılık Geometrisi

Döngüsel bağımlılık, bir parçanın başka bir parçaya, onun da dolaylı olarak ilk parçaya bağımlı olmasıdır:



Sistem kapalı bir döngü oluşturur; bu durum test etmeyi, hata ayıklamayı ve yeniden düzenlemeyi zorlaştırır.

Sistem Sağlığının Geometrik Göstergeleri

Bir sistemin sağlıklı olup olmadığını çoğu zaman geometrisinden anlamak mümkündür; çünkü sağlıklı sistemler belirli yapısal özellikler gösterir:

- yüksek bağdaşıklık (cohesion),
- düşük bağlaşım (coupling),
- net akış yönleri,
- lokal değişim,
- tekrarın azalması,
- hiyerarşik organizasyon,
- anlaşılır bağımlılık grafiği,
- açık sınırlar,

- deterministik akış,
- testlerle sabitlenmiş davranışlar.

Bu özellikler teknik kalite göstergeleri olduğu kadar geometrik göstergeler olarak da okunabilir.

Yüksek Bağdaşıklık

Yüksek bağdaşıklık, bir modülün kendi içinde anlamlı ve tutarlı bir sorumluluğa sahip olmasıdır. Bir dosya veya bileşen yalnızca birbiriyle ilişkili işleri yapıyorsa bağdaşıklık yüksektir. Bağdaşıklık düşükse, aynı parça birbirinden kopuk birçok sorumluluk taşır ve sistemin geometrisi bozulur.

Düşük Bağlaşım

Düşük bağlaşım, sistem parçalarının birbirine gereğinden fazla bağımlı olmamasıdır. Bağlaşım yükseldikçe değişikliklerin etki alanı genişler; bir modüldeki küçük bir değişiklik başka modülleri beklenmedik biçimde etkileyebilir.

Net Akış Yönleri

Net akış yönleri, verinin ve kontrolün sistem içinde hangi yönde ilerlediğinin anlaşılabilir olmasıdır. Arayüzden facade'a, facade'dan servise, servisten dış sisteme giden tek yönlü bir akış, izlenebilir bir geometri üretir.

Lokal Değişim

Lokal değişim, bir özellik veya hata düzeltmesi için yapılan değişikliğin sınırlı bir bölgede kalabilmesidir. Sağlıklı sistemlerde lokal değişiklikler küresel bozulmalara neden olmaz; bu, mimarinin iyi sınırlandığını gösterir.

Anlaşılır Bağımlılık Grafiği

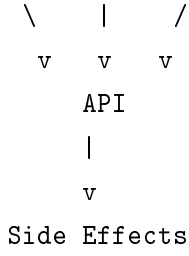
Bağımlılık grafiği, sistemde hangi parçanın hangi parçaya bağlı olduğunu gösterir. Grafik okunabilir ise sistemin zihinsel modeli kurulabilir; grafik karmaşık, döngüsel ve kontrolsüz ise sistemin davranışı tahmin edilemez hale gelir.

Mevcut Geometriden Hedef Geometriye

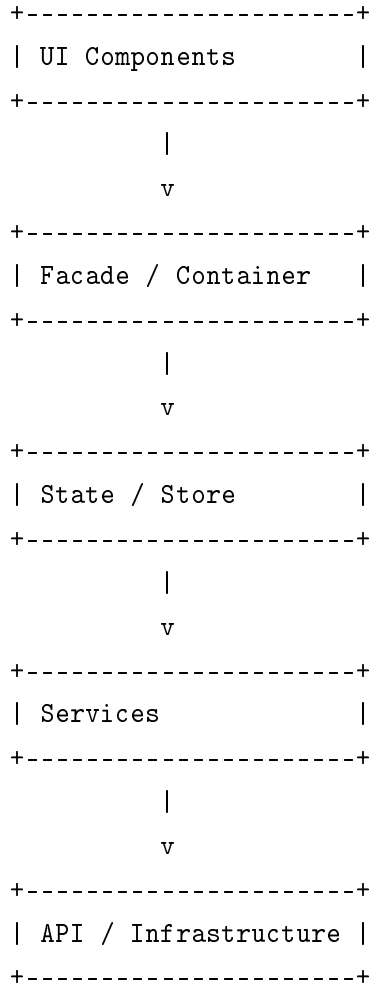
Bir kod tabanı analiz edilirken iki ayrı temsil oluşturmak faydalıdır: mevcut geometri ve hedef sağlıklı geometri.

Mevcut geometri tipik olarak şu görünümü alabilir:

```
    UI
   / | \
  v  v  v
Service Store Helper
```



Bu temsil, veri akışının dağınık olduğunu ve birçok parçanın doğrudan dış sistem veya yan etkilerle temas ettiğini gösterir. Hedef sağlıklı geometri ise şu biçimde tanımlanabilir:



Hedef geometride akış nettir: arayüz doğrudan dış sistemle konuşmaz, yan etkiler belirli bir katmanda toplanır ve durum sahipliği belirginleşir. Bu iki temsil arasındaki fark, yeniden düzenleme çalışmasının yol haritasını oluşturur.

Kod Geometrisinin Analizi

Bir kodun geometrisi, dosyaların fiziksel konumundan çok modüller arasındaki ilişkilerle ilgilidir ve şu sorularla anlaşılabilir:

- Ana modüller nelerdir?

- Veri akışı nereden nereye gider?
- Hangi bileşen, servis veya dosya merkezî konumdadır?
- Hangi parçalar birbirine aşırı bağlıdır?
- Tek yönlü veri akışı var mıdır?
- Döngüsel bağımlılıklar var mıdır?
- Durum sahipliği net midir?
- Sistem ağaç, akış, merkez, ağ, katmanlı veya düğümlenmiş formlardan hangisine benzetilmektedir?

Bu sorular, kodun yalnızca çalışıp çalışmadığını değil, nasıl bir yapısal forma sahip olduğunu ortaya koyar. Bu analiz yapay zekâ araçlarıyla da yürütülebilir; ancak bunun için modelden yalnızca özet istemek yeterli değildir. Modelden bağımlılıkları, veri akışını ve mimari formu çıkarması; mevcut ve hedef geometriyi ayrı temsiller olarak üretmesi; en riskli bağlantıları gerekçelendirmesi ve dosya bazında yeniden düzenleme önerileri sunması istenmelidir. Değerlendirme şeması şu boyutları kapsamalıdır:

+-----+				
Code Geometry				
+-----+				

v	v	v	v	v
Data Flow	Dependency	State	Side	Module
	Direction	Owner	Effects	Shape
v	v	v	v	v
clear?	one-way?	clear?	isolated?	healthy?

Bu değerlendirme, kodun yalnızca bugün çalışıp çalışmadığını değil, uzun vadede yaşayıp yaşayamayacağını anlamaya yarar.

Kod Sadece Metin Değildir

Modern yazılım mimarisi araçlarının bağımlılık grafiği, çağrı grafiği, modül grafiği ve çalışma zamanı izleri gibi görselleştirmeler sunması tesadüf değildir; çünkü yazılım yalnızca metin değildir. Kod aynı zamanda bir yapıdır:

Kod sadece metin değildir. Kod; bir şekil, bir ağ ve bir akış sistemidir.

Bu nedenle yazılımı yalnızca satır satır okumak bazen yeterli olmaz; sistemin nasıl bağlandığını, hangi yönlerde aktığını, hangi parçaların merkezileştiğini ve hangi bölgelerin çamurlaştığını görmek gerekir.

Kodun geometrisi, yapay zekâ destekli geliştirme açısından ayrıca önemlidir. Yapay zekâ; sistemdeki örüntüleri, isimlendirmeleri, sınırları ve bağımlılık ilişkilerini kullanarak öneri üretir. Sistemin geometrisi sağlıklıysa model doğru örüntüyü sürdürebilir; geometri bozuksa model yanlış örüntüleri doğruymuş gibi çoğaltabilir. Dolayısıyla mimari kalite, yalnızca insan okunabilirliği açısından değil, makineyle birlikte akıl yürütebilme açısından da belirleyicidir.

Sonuç

“Bilgi geometrisi” resmi bir bilgisayar bilimi terimi olmayabilir; ancak bir sistemde bilginin nasıl organize olduğunu, nasıl aktığını ve nasıl bağlandığını ifade etmek için güçlü ve açıklayıcı bir metafordur. Bu metafor özellikle yazılım mimarisi için değerlidir; çünkü yazılım sistemleri yalnızca kod satırlarından değil, bağımlılık grafiklerinden, veri akışlarından, sınırlardan, durum kaynaklarından, testlerden ve alan ilişkilerinden oluşur.

Bu çalışmanın merkezi sezgisi şudur:

Karmaşıklığın şekli vardır; ve bir sistemin sağlıklı olup olmadığı çoğu zaman bu şekle bakılarak anlaşılabilir.

Akışlar net, bağımlılıklar kontrollü, merkezler dengeli ve katmanlar belirginse sistem sağlıklıdır. Bağlantılar rastgele, döngüler gizli, merkezler aşırı şişmiş ve veri akışı belirsizse sistemin geometrisi bozulmuştur. Özetle:

Kodun geometrisi bozuksa sistemin anlamı dağınık; kodun geometrisi sağlıklıysa hem insan hem yapay zekâ sistem üzerinde daha doğru düşünebilir.